CleanCrawl: GNOMI Hackathon Project 2026

A respectful crawler for news articles, blog posts, and wiki-style pages

Problem Statement

How can we build a reliable article crawler that collects clean, unique, high-quality content from news sites, blogs, and wiki-style pages despite anti-bot systems, duplicate URLs, messy HTML, JavaScript rendering, and low-quality page layouts?

Project Scenario

You are building a crawler for a news, blog, and wiki knowledge system. The goal is to collect article-style and informational pages that can later be used for search, summarization, recommendations, or retrieval-augmented generation (RAG).

Target Page Types

- News articles
- Blog posts
- Wiki or documentation pages
- Educational pages
- Long-form informational content
- Financial - Tax documents

Pages to Avoid

- Homepages
- Category pages
- Tag pages
- Search result pages
- Login pages
- Ads and popups
- Author pages
- Comment sections
- Duplicate article copies

Main Challenge

Build a crawler pipeline that can safely fetch pages, identify real content pages, extract clean article text, remove duplicates, handle messy HTML, avoid crawler traps, and score final page quality.

1. Fetch Pages Respectfully

- Respect robots.txt
- Use rate limits per domain
- Detect blocked pages
- Detect CAPTCHA, Cloudflare, and anti-bot responses
- Retry failed requests safely
- Avoid aggressive crawling behavior

```
{
  "url": "https://example-news.com/article/ai-policy",
  "status": "blocked",
  "blocked_reason": "cloudflare_or_captcha",
  "should_retry": false
}
```

2. Detect Article-Like Pages

The crawler should classify whether a page is useful content before spending too much time extracting and storing it.

Useful examples:

```
/news/2026/ai-regulation-explained
/blog/how-search-engines-work
/wiki/Web_crawler
/docs/crawling-best-practices
```

Not useful examples:

```
/tag/ai
/category/news
/search?q=crawler
/login
/author/john
/page/20
```

```
{
  "page_type": "news_article",
  "is_content_page": true,
  "confidence": 0.93
}
```

3. Extract Clean Article Content

The system should remove noisy page elements and extract the important article data.

- Title
- Author
- Publish date
- Main text
- Headings
- Canonical URL
- Useful images
- Source domain
- Language
- Article summary

The crawler should remove navigation menus, ads, cookie banners, newsletter popups, related article widgets, social share buttons, comments, and footer links.

```
{
  "url": "https://example.com/news/ai-crawlers",
  "canonical_url": "https://example.com/news/ai-crawlers",
  "title": "How AI Crawlers Are Changing the Web",
  "author": "Jane Smith",
  "published_date": "2026-06-01",
  "page_type": "news_article",
  "language": "en",
  "main_content": "Clean extracted article text...",
  "summary": "The article explains how AI crawlers are affecting publishers...",
  "quality_score": 0.89
}
```

4. Handle Duplicate and Near-Duplicate Pages

News, blog, and wiki-style content often appears multiple times because of tracking parameters, AMP pages, printer pages, mobile pages, syndicated copies, copied blog posts, translated pages, or the same article under different category URLs.

```
/article/ai-news
/article/ai-news?utm_source=twitter
/amp/article/ai-news
/mobile/article/ai-news
/print/article/ai-news
```

The crawler should detect duplicates using canonical URLs, normalized URLs, title similarity, content hashing, near-duplicate text similarity, and publish date comparison.

```
{
  "is_duplicate": true,
  "duplicate_of": "https://example.com/article/ai-news",
  "duplicate_reason": "same_canonical_url_and_similar_text"
}
```

5. Deal with Bad and Messy HTML

Many content pages have broken or noisy HTML. The crawler should still extract useful text from broken tags, nested divs, missing metadata, lazy-loaded images, hidden content, badly structured blog templates, and wiki pages with tables, references, and sidebars.

```
{
  "html_quality": "messy",
  "problems_detected": [
    "broken_html",
    "large_navigation_noise",
    "missing_author",
    "cookie_banner_removed"
  ]
}
```

6. Avoid Crawler Traps

The crawler should avoid endless crawling through calendars, archive pages, tag combinations, search pages, infinite pagination, filter URLs, comment pagination, and sort parameters.

```
{
  "url": "https://example.com/archive/2021/05/page/400",
  "crawl_decision": "skip",
  "reason": "pagination_depth_too_high"
}
```

Final Challenge Statement

Build an intelligent article crawler that can discover, clean, deduplicate, and score news articles, blog posts, and wiki-style pages from the messy real web while respecting website rules and detecting anti-bot blocks.

Suggested Judging Criteria

Area	What to Measure
Crawl safety	Respects robots.txt, uses rate limits, and avoids aggressive retries.
Anti-bot detection	Identifies CAPTCHA, Cloudflare, blocked pages, fake pages, and suspicious responses.
Content extraction	Extracts clean title, date, author, body text, headings, and metadata.
Duplicate detection	Removes URL duplicates and near-duplicate articles.
Messy HTML handling	Extracts useful content from noisy, broken, or inconsistent page structures.

Page classification	Separates articles, blogs, and wiki pages from tag, category, login, search, and archive pages.
Quality scoring	Ranks pages by usefulness, freshness, completeness, uniqueness, and cleanliness.
Scalability	Uses a queue, retry system, domain limits, logs, and monitoring-friendly design.

Bonus Features

- Showing crawl success rate, blocked pages, duplicates removed, and extraction quality
- Browser-based fallback for JavaScript-rendered pages
- Messy HTML cleanup or field extraction
- Automatic detection of anti-bot pages
- Smart recrawl scheduler for frequently changing pages
- Content quality confidence score with explainable reasons